



LarvaLens

Implementation Plan Eight-Hour Build Sequence

The exact build order for setup, database/auth, backend, real inference, frontend, testing, deployment, feature freeze and final completion criteria.

Prepared for	Mohit S. and the LarvaLens team
Build format	Vibe-coding source-of-truth document 06
Version	1.0 - 26 August 2026



Vibe Document 06 - Implementation Plan

Document ID: 06_IMPLEMENTATION_PLAN.md Status: Source of truth

11.1 Eight-hour Lap 3 sequence

Time	Phase	Deliverable and done criterion
0:00-0:30	Foundation	Monorepo folders, env example, lint/typecheck/test scripts, no secrets committed.
0:30-1:10	Database/Auth	Migration and RLS applied; real signup/login creates a profile; unauthorized reads fail.
1:10-2:10	API shell	Health/model status/upload validation/scan creation implemented with typed errors. Missing models returns 503.
2:10-3:00	Web shell	Routes, auth guard, scan form and real API client work on desktop/mobile.
3:00-4:15	Model integration	e050 detector + verifier load once; one real clip creates real tracks/result.
4:15-5:15	Realtime/results	Real stage updates reach processing page; result/evidence/history render DB rows.
5:15-6:00	Map/review	Aggregated map and reviewer flow work with RLS.
6:00-7:00	Error/quality pass	Model missing, corrupt video, low quality, network loss and unauthorized routes tested.
7:00-8:00	Freeze/deploy/backup	Vercel frontend, reachable inference API, smoke test, screenshots, repo tag and backup recording.

11.2 Phase 1 - setup

- Create the repository tree from the TRD.
- Add .gitignore for .env*, models/active/*.pt, uploads, temporary frames, local DB files and Python/Node caches.
- Add .env.example with names only.
- Configure TypeScript strict mode, ESLint, Prettier/Ruff and pytest.
- Write a README quick start.

Done when both apps start, lint/typecheck pass and no secret/model/video is tracked.

11.3 Phase 2 - database and auth

- Apply the schema in a clean Supabase project.
- Enable Realtime for scans.
- Create private storage buckets.
- Test a real signup, profile trigger and login.
- Verify field worker cannot read another user's scan or write result fields.
- Assign reviewer/admin manually through a trusted SQL/admin workflow.

Done when permission tests pass with two users and one reviewer.

11.4 Phase 3 - backend foundation

- Add settings validation and CORS allowlist.
- Add request ID middleware and structured error model.
- Implement JWT verification and current-user dependency.
- Implement model manifest parser, hash verifier and lifespan loader.
- Implement media validation and temporary-file cleanup.
- Implement repository/service layers for scans, tracks, events and storage.

Done when model status reports actual readiness and missing files cause a stable 503.

11.5 Phase 4 - real analysis path

- POST creates the real scan and storage object.
- Worker updates real stages.
- Integrate Lap 2 reference analyzer behind `InferenceService`.
- Persist tracks and final result atomically.
- Generate evidence images/video only if time permits; raw JSON evidence is mandatory.
- Record model SHA-256 values on every scan.

Done when a real clip survives upload -> inference -> database -> result without manual intervention.

11.6 Phase 5 - frontend

- Implement auth and protected routes.
- Build scan capture/upload and location consent.
- Connect 202 response to processing route.
- Subscribe to real row updates; refetch on reconnect.
- Build result, track evidence, history and map.
- Build reviewer pages only after the owner flow works.

Done when the browser displays the exact DB/API result and never a local placeholder.

11.7 Phase 6 - tests

Backend tests:

- Missing/invalid JWT -> 401.
- Wrong owner -> 404/403 without leaking existence.
- Missing model/hash mismatch -> 503.
- Oversized/unsupported/corrupt video -> 422/413.
- Valid scan -> 202 and real queued row.
- Inference exception -> failed row and safe error.
- Completion -> tracks and scan final state consistent.

Frontend tests:

- Auth hydration/redirect.
- File validation.
- Model-not-ready error.
- Realtime reconnect/refetch.
- Completed and retake-required screens.
- Empty history/map.
- Reviewer route authorization.

11.8 Phase 7 - deployment

- Deploy apps/web to Vercel.
- Run API on the demo laptop/LAN or a persistent Python host with required model/storage access.
- Configure exact origins and HTTPS endpoint if internet-facing.
- Update VITE_API_BASE_URL, rebuild web and smoke test from the judge-facing phone.
- Keep a fully local fallback: browser/PWA on LAN -> local API -> Supabase if internet works, or prepared processed backup if network fails.

11.9 Feature freeze

At Hour 7, do not add new features. Only fix blocking bugs, model paths, thresholds already justified by validation, mobile overflow, credentials/configuration and demo reliability.

11.10 Final done criteria

- Two required model hashes validate.
- Real user can complete the core flow.
- No production mock data is present.
- RLS prevents cross-user access.
- Realtime reflects actual stage writes.
- Model missing and bad media fail honestly.
- Held-out field metrics and limitations are available for the pitch.
- Repo, model manifest, screenshots, short backup video and deployment URLs are saved.